

CAPSTONE PROJECT ARCHITECTURE DOCUMENT

THE CREW

Design, Architecture, and Mobile Implementation of a Real-Time Gaming Community and Spatial Communication Platform.

PROJECT CONTRIBUTORS

Pranav Kale

Backend Architecture, Database Design & Real-Time Protocol Engineering

R Virshin

Mobile Client Engineering, State Architecture & Mobile UI/UX Design

PROJECT TITLE: The Crew (Gaming Community & Communication Platform)

TECHNOLOGIES: Flutter 3.x, Dart 3.x, Node.js, Express, Socket.IO, SQLite (WAL)

TARGET VIEWPORT: Mobile Smartphone (390 × 844 pt @ 2x DPR / 780 × 1688 px)

DOCUMENT VERSION: 1.0.0 (Final Submission)

DATE OF SUBMISSION: September 2026

SECTION 1.0

Project Abstract & Declaration of Originality

1.1 PROJECT ABSTRACT

Digital gaming squads, competitive esports teams, and modern creative communities rely heavily on real-time communication systems that provide immediate responsiveness, low-latency audio signaling, and organized text spaces. Legacy communication clients often suffer from platform bloat, intrusive tracking, slow startup times, and complex navigation structures that diminish mobile usability.

The Crew is engineered from first principles as an integrated cross-platform mobile client and real-time backend service. Built using Flutter and Dart on the client side, backed by an Express and Socket.IO microservice architecture with an embedded high-performance SQLite engine, the system delivers sub-second message delivery, immediate emoji reactions, live presence heartbeats, and private direct messaging in a clean, ergonomic mobile viewport.

1.2 KEY PROJECT OBJECTIVES

- **Mobile Ergonomics:** Deliver a native-grade 390x844 pt smartphone experience with responsive layouts.
- **Event-Driven State:** Establish bidirectional WebSocket channels for immediate message delivery and presence.
- **Zero-Configuration Startup:** Embed an autonomous SQLite WAL database engine with automatic schema migrations.
- **Granular Security:** Enforce bcrypt password hashing (10 salt rounds) and signed JSON Web Tokens (JWT).

1.3 DECLARATION OF ORIGINALITY

We hereby declare that this project titled "**The Crew: Real-Time Gaming Community & Spatial Communication Platform**" is an original work designed, implemented, and submitted by:

- **Pranav Kale**
Backend Architecture, Database Design & Socket Protocols
- **R Virshin**
Mobile Application Engineering, UI/UX & State Architecture

1.4 ENGINEERING DELIVERABLES & MILESTONES

MILESTONE	DELIVERABLE	VERIFICATION CRITERIA
Client Application	Flutter 3.x Mobile App	15 screens, GoRouter shell navigation, Provider state management, 0 linter issues.
API Gateway	Express REST Engine	18+ endpoints covering Auth, Servers, Channels, Messages, DMs, Voice, and Notifications.
Real-Time Pipeline	Socket.IO Event Cluster	Sub-second dispatch for live chat, typing indicators, reactions, and audio states.
Data Persistence	SQLite (WAL Mode)	9 relational tables with foreign keys and cascade delete rules, auto-seeded on boot.

SECTION 2.0

High-Level System Architecture

2.1 SYSTEM ARCHITECTURE OVERVIEW

The system follows a decoupled 3-tier architecture designed for scalability, maintainability, and low-latency interaction:

1. **Presentation Tier (Mobile Client):** Written in Dart using Flutter. Handles all rendering, gesture interpretation, and localized state caching. Communicates via REST for transactional state changes and WebSockets for real-time synchronization.
2. **Application Tier (API Gateway & Real-Time Server):** Built with Node.js, Express, and Socket.IO. Manages authentication, business logic validation, route protection, and socket room membership.
3. **Data Tier (Persistence):** Structured relational storage powered by SQLite using better-sqlite3. Write-Ahead Logging (WAL) is enabled to support concurrent read operations alongside write transactions.

2.2 NETWORK TOPOLOGY & EVENT FLOW

Socket connections are authenticated at connection time using Bearer JWT tokens. Upon handshake, sockets join granular rooms:

- `user_<id>`: Private room for direct messages and direct mentions.
- `channel_<id>`: Text channel room for chat messages, reactions, and typing states.
- `voice_<id>`: Voice room for participant joined, left, speaking, and mute events.

This room-based segmentation avoids broadcast overhead across non-participating clients, ensuring low resource usage.

2.3 THREE-TIER COMPONENT DIAGRAM

ARCHITECTURE TIER	CORE TECHNOLOGIES	PRIMARY RESPONSIBILITIES
Tier 1: Client Layer	Flutter 3.x, Dart 3.x, GoRouter, Provider, SharedPreferences	UI rendering, user input handling, offline session restoration, socket message consumption, dynamic theme engine.
Tier 2: Service Layer	Node.js 25, Express.js, Socket.IO 4.x, bcryptjs, jsonwebtoken	HTTP API endpoints, JWT token issuance and verification, WebSocket event broadcast, room lifecycle management.
Tier 3: Database Layer	SQLite 3 (WAL Mode), better-sqlite3 engine	Relational data persistence, foreign key enforcement, transaction management, auto-seeding engine on startup.

SECTION 3.0

Screen Catalog & Specification Index

3.0 COMPLETE 15-SCREEN MOBILE CATALOG				
#	SCREEN TITLE	ROUTE PATH	CATEGORY	ARCHITECTURAL PURPOSE
01	Splash & Protocol Initialization	/splash	Lifecycle & Initialization	The initial application gateway that boots the Flutter runtime, verifies backend serv...
02	Welcome & Onboarding Portal	/welcome	Brand & Architecture Introduction	The primary user onboarding screen introducing users to The Crew's spatial audio, 4K ...
03	Authentication & Quick Demo Login	/login	Security & Access Control	Secure login screen supporting traditional password authentication, device persistenc...
04	Account Registration & Personalization	/create-account	User Provisioning	Account creation flow enabling new members to register unique usernames, custom displ...
05	Servers Hub & Live Channel Chat	/servers	Core Messaging & Channels	The central hub of The Crew featuring a multi-server vertical navigation rail, channe...
06	Spatial Voice Channel & Live Stage	/servers/voice	Audio & Live Stage	Low-latency spatial audio stage featuring high-bitrate Opus stream status, live scree...
07	Server Member Directory & Hierarchy	/servers/members	Directory & Permissions	Hierarchical server member directory categorizing community members by permission lev...
08	Direct Messages Inbox & Active Squad	/messages	Direct Messaging	Private messaging hub featuring a top horizontal "Active Now" squad story rail, searc...
09	1-on-1 Direct Message Conversation	/messages/chat/:userId	Private Chat Thread	Full-screen private direct chat interface between two users, supporting real-time mes...
10	Community Discovery & Exploration	/discover	Public Discovery	Public server discovery portal allowing users to explore featured gaming guilds, musi...
11	Community Creation & Setup	/discover/create	Server Provisioning	Server provisioning wizard allowing community creators to launch custom hubs,

SECTION 3.0 (CONT.)

Client State Management & Routing Architecture

3.0.1 INDEXED SHELL ROUTING (GOROUTER)

Navigation is structured using GoRouter and StatefulShellRoute.indexedStack. This architecture ensures that each primary navigation branch maintains its independent navigation state and scroll offset in memory when switching tabs.

Full-screen modal routes (such as /messages/chat/:userId and /servers/voice) target _rootNavigatorKey, allowing them to render cleanly above the bottom navigation bar without layout jumps.

3.0.2 DOMAIN STATE PROVIDERS

State is managed using the Provider package across 6 decoupled domain notifiers:

- **AuthProvider:** Token persistence, user profile, login, and registration.
- **ServerProvider:** Server memberships, channel listings, and member roles.
- **ChatProvider:** Active channel message stream and emoji reaction state.
- **DmProvider:** Direct message conversations and active squad members.
- **VoiceProvider:** Voice channel participant states, mute, and speaking flags.

3.0.3 SHELL ROUTE HIERARCHY SPECIFICATION

BRANCH	ROOT ROUTE	SUB-ROUTES	ASSIGNED SCREEN WIDGETS
Branch 0	/servers	/members, /voice	MainChatScreen, MemberListScreen, VoiceChannelScreen
Branch 1	/messages	/chat/:userId	DmInboxScreen, DmChatScreen
Branch 2	/discover	/create	CommunityDiscoveryScreen, CreateCommunityScreen
Branch 3	/activity	–	NotificationsScreen
Branch 4	/profile	/settings, /theme	UserProfileScreen, SettingsScreen, CustomizeThemeScreen

SECTION 3.1 • SCREEN SPECIFICATION

Splash & Protocol Initialization

/splash

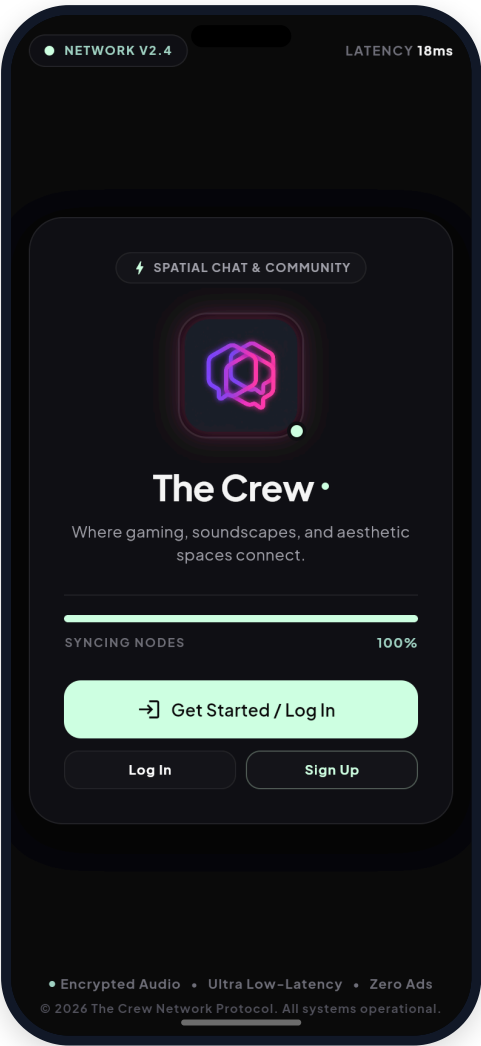


Figure 1: Mobile viewport (Splash & Protocol Initialization, 390 × 844 pt).

1. FUNCTIONAL SCOPE & USER INTENT

The initial application gateway that boots the Flutter runtime, verifies backend service health, and restores persistent authentication credentials from local storage.

2. UI COMPONENTS & VISUAL ANATOMY

- Top network telemetry bar displaying active protocol version (v2.4) and live latency ping (18ms).
- Centered geometric chat bubble brand logo with subtle state feedback.
- Core system identity statement: "Where gaming, soundscapes, and aesthetic spaces connect."
- Node synchronization progress bar indicating runtime bootstrap status (100%).
- Primary CTA button "Get Started / Log In" alongside direct secondary access buttons.

3. TECHNICAL & ARCHITECTURAL MAPPING

Route & Guard:	/splash (Public / Boot Entrypoint)
State Providers:	AuthProvider (restoreSession)
REST API Calls:	GET /api/health (status ping & latency telemetry)
Socket.IO	Initiates WebSocket handshake if valid JWT
Events:	found in cache
Database Tables:	users (session validation)

4. USER INTERACTION FLOW & STATE VALIDATION

Automatically queries local device storage. If a valid JWT session exists, the app transitions seamlessly to the main server hub without requiring user intervention. If first-time launch, presents authentication choices.

SECTION 3.2 • SCREEN SPECIFICATION

Welcome & Onboarding Portal

/welcome

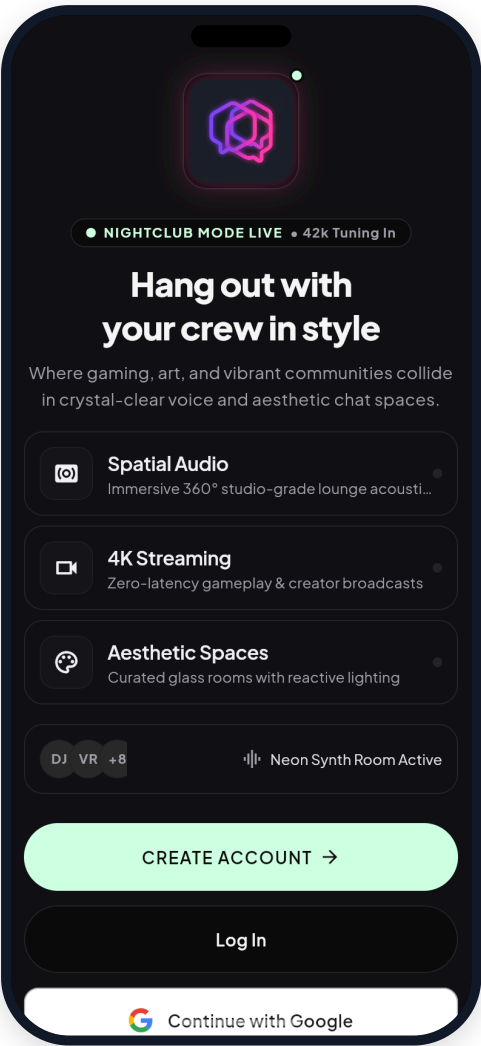


Figure 2: Mobile viewport (Welcome & Onboarding Portal, 390 × 844 pt).

1. FUNCTIONAL SCOPE & USER INTENT

The primary user onboarding screen introducing users to The Crew’s spatial audio, 4K screen streaming, and aesthetic glass room capabilities.

2. UI COMPONENTS & VISUAL ANATOMY

- Interactive "Nightclub Mode Live" pill with real-time listener counter badge (42k Tuning In).
- High-contrast headline typography: "Hang out with your crew in style".
- Feature showcase cards detailing Spatial Audio (360° lounge acoustics), 4K Zero-latency Streaming, and Aesthetic Glass Rooms.
- Active voice lobby badge featuring stacked avatar circles (DJ, VR, +8) and graphic equalizer icon.
- Full-width primary action "CREATE ACCOUNT" button and secondary outlined "Log In" button.

3. TECHNICAL & ARCHITECTURAL MAPPING

Route & Guard:	/welcome (Public / Welcome Flow)
State Providers:	NavigationService, AuthProvider
REST API Calls:	Static local feature definitions with dynamic telemetry pills
Socket.IO	Passive connection listener
Events:	
Database Tables:	N/A

4. USER INTERACTION FLOW & STATE VALIDATION

Presents an intuitive overview of platform capabilities with immediate routing triggers for both new account registration and existing user login.

SECTION 3.3 • SCREEN SPECIFICATION

Authentication & Quick Demo Login

/login

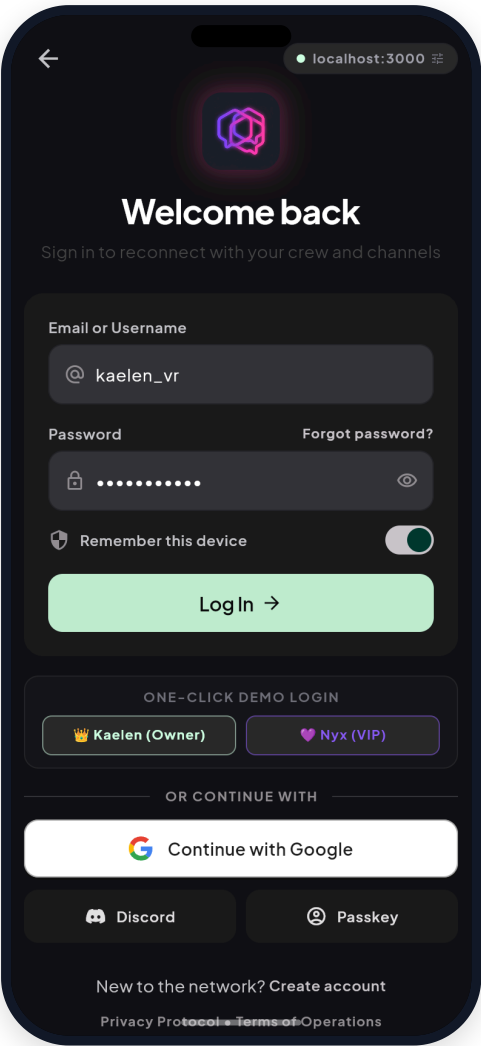


Figure 3: Mobile viewport (Authentication & Quick Demo Login, 390 × 844 pt).

1. FUNCTIONAL SCOPE & USER INTENT

Secure login screen supporting traditional password authentication, device persistence, social OAuth, and an instant one-click developer quick-login dock.

2. UI COMPONENTS & VISUAL ANATOMY

- Standardized form input fields for Username/Email and Password with toggleable visibility.
- Remember this device toggle switch with animated state persistence.
- One-Click Demo Login dock featuring instant persona switches: "Kaelen (Owner)" and "Nyx (VIP)".
- Google Sign-In integration and third-party authentication shortcuts (Discord, Passkey).
- Top navigation bar with back arrow and branding logo.

3. TECHNICAL & ARCHITECTURAL MAPPING

Route & Guard:	/login (Public / Authentication Form)
State Providers:	AuthProvider (login, quickLogin, googleLogin)
REST API Calls:	POST /api/auth/login, POST /api/auth/quick-login, POST /api/auth/google
Socket.IO Events:	Authenticates socket connection with JWT Bearer handshake
Database Tables:	users (credential check & bcrypt hash comparison)

4. USER INTERACTION FLOW & STATE VALIDATION

Facilitates fast evaluation and credentialed access. The one-click demo pills bypass manual typing for rapid feature inspection while retaining full server-side JWT verification.

SECTION 3.4 • SCREEN SPECIFICATION

Account Registration & Personalization

/create-account

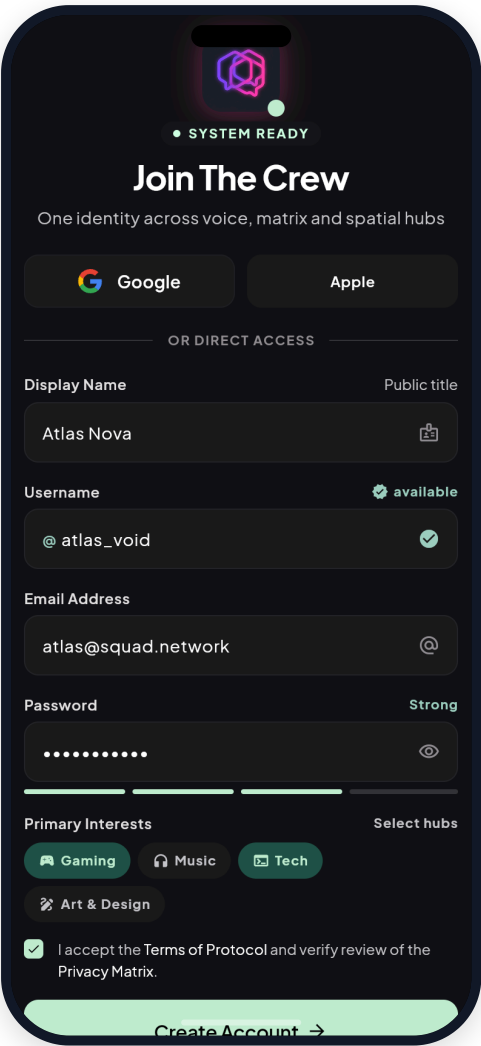


Figure 4: Mobile viewport (Account Registration & Personalization, 390 × 844 pt).

1. FUNCTIONAL SCOPE & USER INTENT

Account creation flow enabling new members to register unique usernames, custom display handles, email verification, and select gaming interest tags.

2. UI COMPONENTS & VISUAL ANATOMY

- Multi-step user profile inputs: Display Name, Unique Handle (@username), Email, and Password.
- Dynamic input validation indicators providing real-time feedback on handle availability.
- Interactive interest chips (Gaming, Tech, Music, Anime, Esports) stored in user profile.
- Terms of Service and Community Guidelines consent checkbox.
- High-contrast "COMPLETE REGISTRATION" button.

3. TECHNICAL & ARCHITECTURAL MAPPING

Route & Guard:	/create-account (Public / User Registration)
State Providers:	AuthProvider (register)
REST API Calls:	POST /api/auth/register
Socket.IO	Emits presence:updated event to announce new
Events:	user join
Database Tables:	users (inserts new user record with default status and bio)

4. USER INTERACTION FLOW & STATE VALIDATION

Validates input fields synchronously on the client and server. Upon successful account creation, it immediately initializes the authenticated session.

SECTION 3.5 • SCREEN SPECIFICATION

Servers Hub & Live Channel Chat

/servers

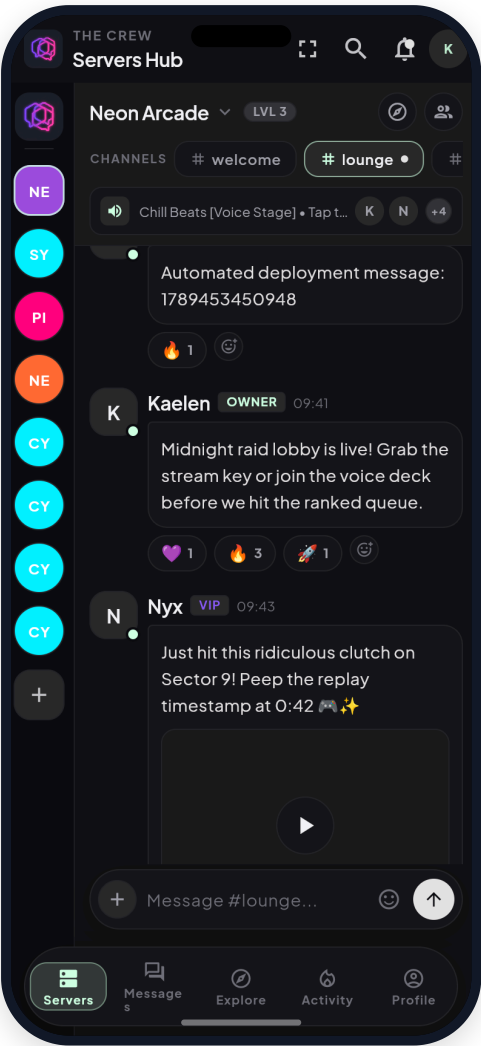


Figure 5: Mobile viewport (Servers Hub & Live Channel Chat, 390 × 844 pt).

1. FUNCTIONAL SCOPE & USER INTENT

The central hub of The Crew featuring a multi-server vertical navigation rail, channel switcher, active voice stage banner, live rich chat messages, and emoji reactions.

2. UI COMPONENTS & VISUAL ANATOMY

- Left vertical server navigation rail with colored server badges, active indicator pill, and "+" add server button.
- Top channel switcher showing current server ("Neon Arcade LVL 3") and horizontal text channels (#welcome, #lounge).
- Persistent voice stage banner showing live participants (K, N, +4) with single-tap join.
- Rich message cards showing user avatar, role badges (OWNER, VIP), timestamps, media attachments, and emoji reactions.
- Bottom message input bar with attachment button, emoji picker trigger, and send button, positioned safely above bottom navigation.

3. TECHNICAL & ARCHITECTURAL MAPPING

Route & Guard:	/servers (Authenticated (Shell Tab 0))
State Providers:	ServerProvider, ChatProvider, AuthProvider
REST API Calls:	GET /api/servers, GET /api/servers/:id/channels, GET /api/channels/:id/messages, POST /api/channels/:id/messages, POST /api/channels/messages/:id/reactions
Socket.IO Events:	channel:join, channel:leave, message:new, reaction:updated, typing:start/stop
Database Tables:	servers, server_members, channels, messages, reactions, users

4. USER INTERACTION FLOW & STATE VALIDATION

Instantaneous switching between servers and channels without reloads. Real-time message streaming with sub-second WebSocket dispatch.

SECTION 3.6 • SCREEN SPECIFICATION

Spatial Voice Channel & Live Stage

/servers/voice

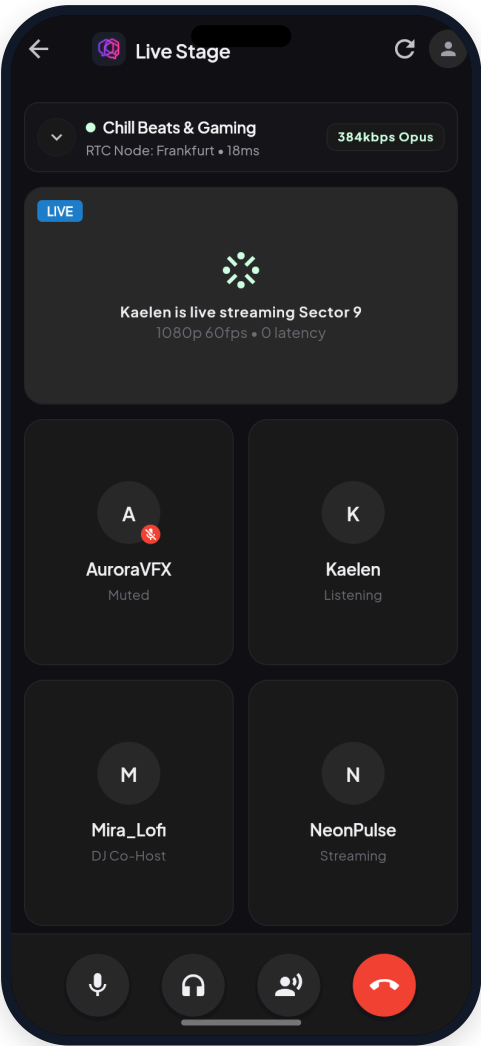


Figure 6: Mobile viewport (Spatial Voice Channel & Live Stage, 390 x 844 pt).

1. FUNCTIONAL SCOPE & USER INTENT

Low-latency spatial audio stage featuring high-bitrate Opus stream status, live screen sharing previews, participant audio state tracking, and quick controls.

2. UI COMPONENTS & VISUAL ANATOMY

- Channel status card: "Chill Beats & Gaming", RTC Node (Frankfurt • 18ms), 384kbps Opus audio badge.
- Live stream preview card with live status indicator and host broadcast text.
- Participant grid showing who is speaking, muted, or streaming.
- Bottom floating audio action dock: Microphone toggle (mute/unmute), Deafen toggle, Participants sheet, and Disconnect button.
- Top navigation bar with back navigation and participant count.

3. TECHNICAL & ARCHITECTURAL MAPPING

Route & Guard:	/servers/voice (Authenticated (Modal / Full Route))
State Providers:	VoiceProvider, ServerProvider
REST API Calls:	GET /api/voice/:channelId, POST /api/voice/:channelId/join, POST /api/voice/:channelId/leave, POST /api/voice/:channelId/toggle-mute, POST /api/voice/:channelId/speaking
Socket.IO Events:	voice:join, voice:leave, voice:joined, voice:left, voice:mute_updated, voice:speaking_updated
Database Tables:	voice_participants, channels, users

4. USER INTERACTION FLOW & STATE VALIDATION

Provides clear visual cues for audio state transitions. Microphone mute and deafen toggles provide immediate tactile and visual feedback.

SECTION 3.7 • SCREEN SPECIFICATION

Server Member Directory & Hierarchy

/servers/members

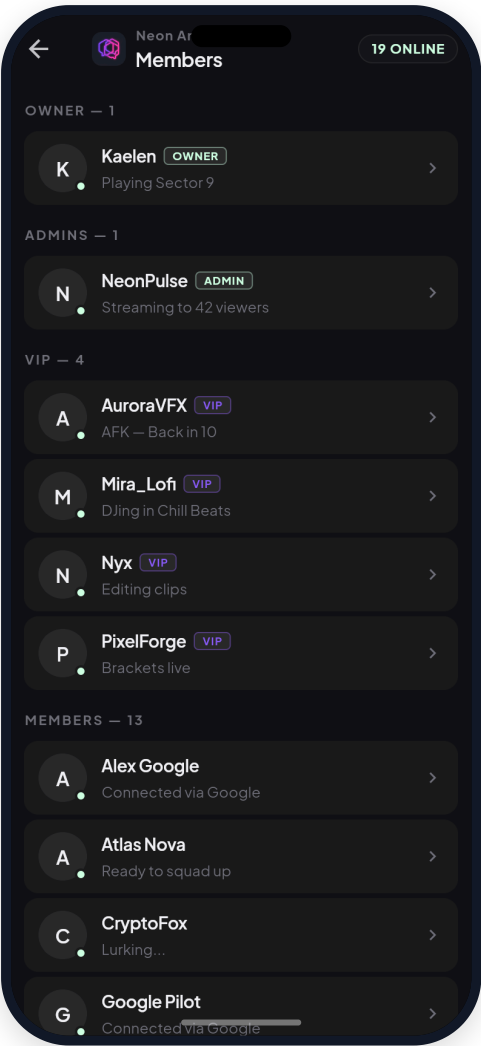


Figure 7: Mobile viewport (Server Member Directory & Hierarchy, 390 × 844 pt).

1. FUNCTIONAL SCOPE & USER INTENT

Hierarchical server member directory categorizing community members by permission level with live status dots, custom game activity, and profile triggers.

2. UI COMPONENTS & VISUAL ANATOMY

- Top app bar showing server name ("Neon Arcade"), Members title, and "19 ONLINE" badge.
- Role-based grouping sections: OWNER (1), ADMINS (1), VIP (4), MEMBERS (13).
- Member cards displaying display name, role pill, live status indicator, and current activity status.
- Tap-to-profile gesture detector allowing direct messaging or role inspection.
- Smooth vertical scrolling with sliver-based performance optimization.

3. TECHNICAL & ARCHITECTURAL MAPPING

Route & Guard:	/servers/members (Authenticated (Server Drawer))
State Providers:	ServerProvider (members list)
REST API Calls:	GET /api/servers/:serverId/members
Socket.IO	presence:updated (live status
Events:	synchronization)
Database Tables:	server_members, users

4. USER INTERACTION FLOW & STATE VALIDATION

Gives server owners and members an organized hierarchy of current community members, distinguishing leadership from regular members.

SECTION 3.8 • SCREEN SPECIFICATION

Direct Messages Inbox & Active Squad

/messages

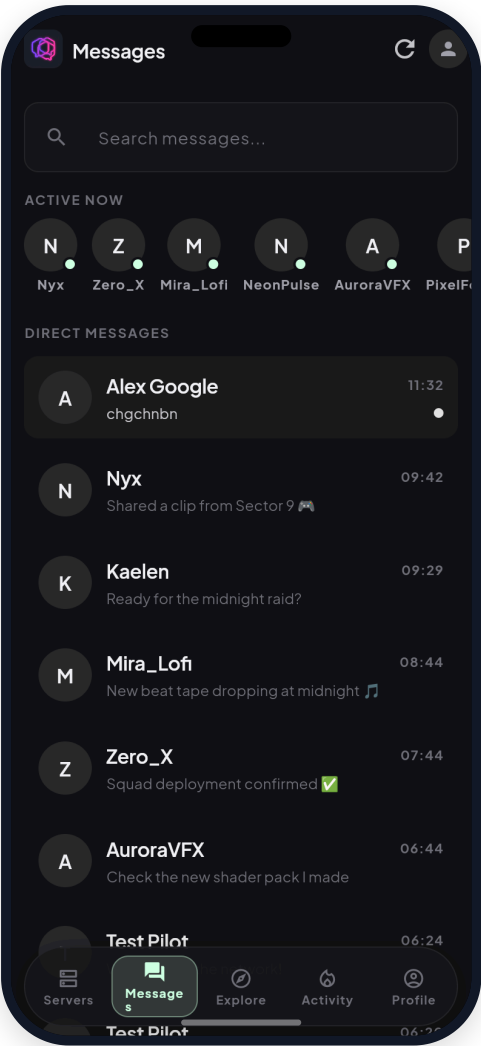


Figure 8: Mobile viewport (Direct Messages Inbox & Active Squad, 390 × 844 pt).

1. FUNCTIONAL SCOPE & USER INTENT

Private messaging hub featuring a top horizontal "Active Now" squad story rail, search bar, and conversation cards with unread notification pills.

2. UI COMPONENTS & VISUAL ANATOMY

- Top app bar with Messages title, manual refresh action, and user avatar shortcut.
- Integrated search field for filtering direct conversations and squad members.
- Horizontal "ACTIVE NOW" stories rail with user avatar, name, and online status dot.
- Conversation list showing partner name, snippet of latest message, timestamp, and unread count badge.
- One-tap navigation into dedicated 1-on-1 private chat threads.

3. TECHNICAL & ARCHITECTURAL MAPPING

Route & Guard:	/messages (Authenticated (Shell Tab 1))
State Providers:	DmProvider, AuthProvider
REST API Calls:	GET /api/dms (fetches active_now users & recent conversations)
Socket.IO Events:	dm:new (updates inbox snippet and bumps unread count in real-time)
Database Tables:	direct_messages, users

4. USER INTERACTION FLOW & STATE VALIDATION

Adopts mobile-native conversation listing patterns. The top rail provides one-tap reach to online friends, while the list below orders active threads chronologically.

SECTION 3.9 • SCREEN SPECIFICATION

1-on-1 Direct Message Conversation

/messages/chat/:userId

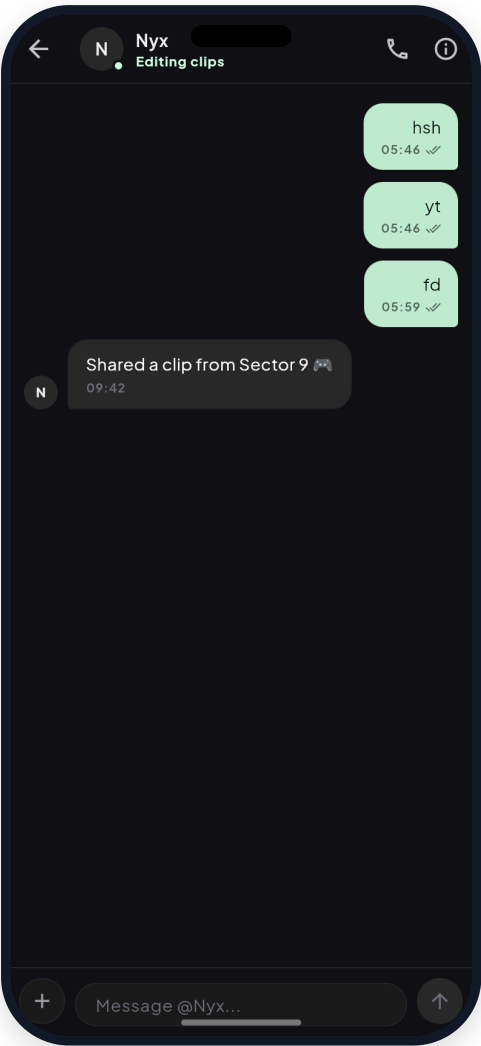


Figure 9: Mobile viewport (1-on-1 Direct Message Conversation, 390 × 844 pt).

1. FUNCTIONAL SCOPE & USER INTENT

Full-screen private direct chat interface between two users, supporting real-time messaging, delivery status indicators, and partner profile inspection.

2. UI COMPONENTS & VISUAL ANATOMY

- Custom app bar featuring partner avatar with online dot, partner name ("Nyx"), status ("Editing clips"), voice call button, and info button.
- Asymmetric chat bubbles: Right-aligned bubbles for the authenticated user; Left-aligned bubbles for the chat partner.
- Delivery timestamp and double-check read status markers.
- Bottom message composition bar with attachment trigger, text field ("Message @Nyx..."), and send button.
- Full back-navigation support with GoRouter integration.

3. TECHNICAL & ARCHITECTURAL MAPPING

Route & Guard:	/messages/chat/:userId (Authenticated (Nested Route))
State Providers:	DmProvider (loadDmThread, sendDm)
REST API Calls:	GET /api/dms/:userId, POST /api/dms/:userId
Socket.IO Events:	dm:new (listens on personal room user_ for instant message reception)
Database Tables:	direct_messages (updates is_read=1 on thread open)

4. USER INTERACTION FLOW & STATE VALIDATION

Private, secure chat thread that automatically marks unread messages as read upon viewing and receives incoming messages without requiring page reloads.

SECTION 3.10 • SCREEN SPECIFICATION

Community Discovery & Exploration

/discover

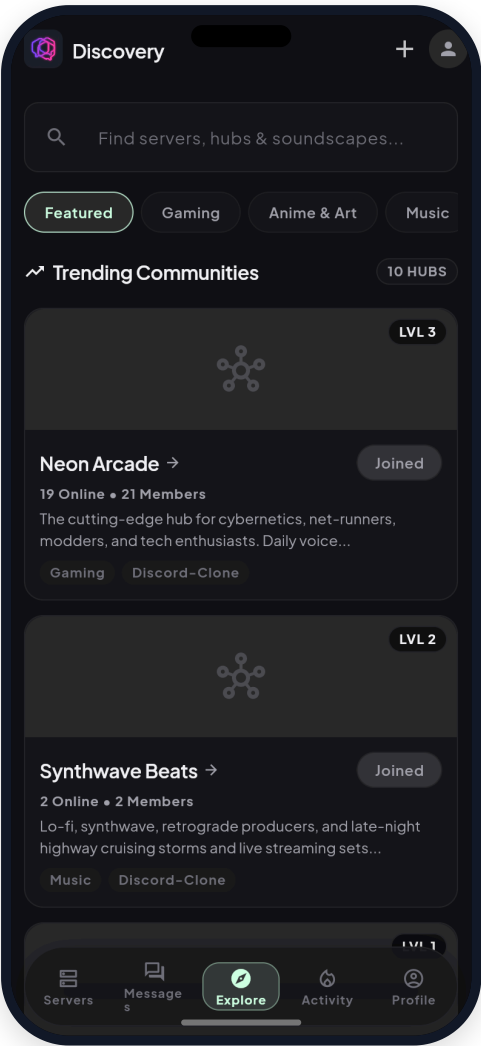


Figure 10: Mobile viewport (Community Discovery & Exploration, 390 × 844 pt).

1. FUNCTIONAL SCOPE & USER INTENT

Public server discovery portal allowing users to explore featured gaming guilds, music hubs, and art communities with rich category filtering and search.

2. UI COMPONENTS & VISUAL ANATOMY

- Top search bar: "Find servers, hubs & soundscapes..." with real-time query debounce.
- Category filter chips: Featured, Gaming, Anime & Art, Music, Tech.
- Trending Communities header displaying total available hubs count ("10 HUBS").
- Rich community cards with server icon, level badge (LVL 3), member metrics ("19 Online • 21 Members"), description, and tags.
- Action button on cards: "Joined" indicator for joined communities or "Join Server" for new communities.

3. TECHNICAL & ARCHITECTURAL MAPPING

Route & Guard:	/discover (Authenticated (Shell Tab 2))
State Providers:	ServerProvider (fetchDiscoverServers, joinServer)
REST API Calls:	GET /api/servers/discover?category=...&search=..., POST /api/servers/:id/join
Socket.IO	Passive socket state
Events:	
Database Tables:	servers, server_members, users

4. USER INTERACTION FLOW & STATE VALIDATION

Enables exploration of public hubs with intuitive category chips and instant join actions that immediately reflect in the user's server rail.

SECTION 3.11 • SCREEN SPECIFICATION

Community Creation & Setup

/discover/create

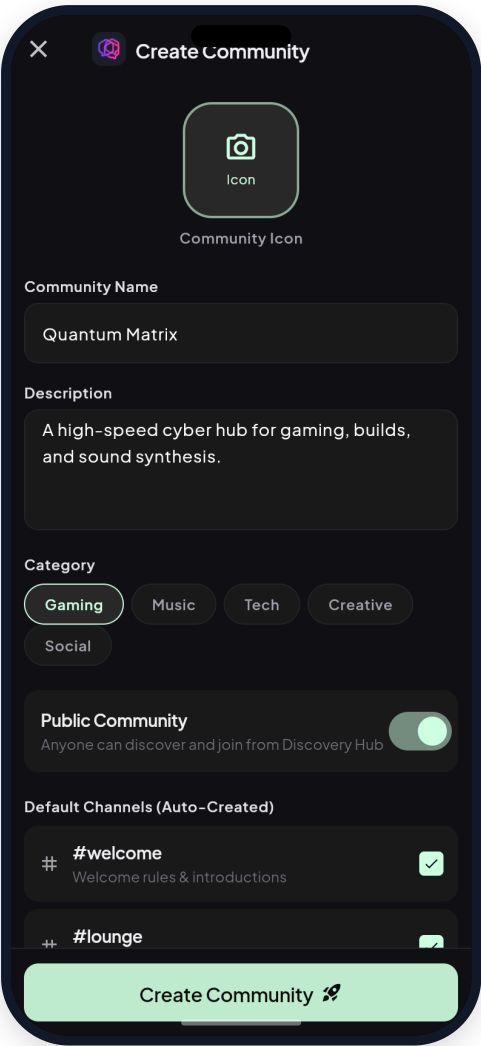


Figure 11: Mobile viewport (Community Creation & Setup, 390 × 844 pt).

1. FUNCTIONAL SCOPE & USER INTENT

Server provisioning wizard allowing community creators to launch custom hubs, select branding colors, configure privacy settings, and auto-provision channels.

2. UI COMPONENTS & VISUAL ANATOMY

- Server Name and Description input fields with helpful placeholder examples.
- Interactive Icon Color Palette picker (Neon Violet, Cyber Mint, Electric Pink, Sunset Orange, Bright Cyan).
- Category selection dropdown (Gaming, Music, Art, Tech, Cozy).
- Public vs. Private Community toggle switch with explanatory privacy caption.
- Large "CREATE COMMUNITY" action button that provisions default channels (#welcome, #lounge, #media-share, Lounge Voice).

3. TECHNICAL & ARCHITECTURAL MAPPING

Route & Guard:	/discover/create (Authenticated (Modal / Route))
State Providers:	ServerProvider (createServer)
REST API Calls:	POST /api/servers
Socket.IO	Emits server:created event to update member
Events:	client rails
Database Tables:	servers (insert), server_members (inserts creator as OWNER), channels (provisions 4 default channels)

4. USER INTERACTION FLOW & STATE VALIDATION

Simplifies server initialization into a single screen, automatically creating essential channel templates and assigning admin roles.

SECTION 3.12 • SCREEN SPECIFICATION

Activity Feed & Real-time Notifications

/activity

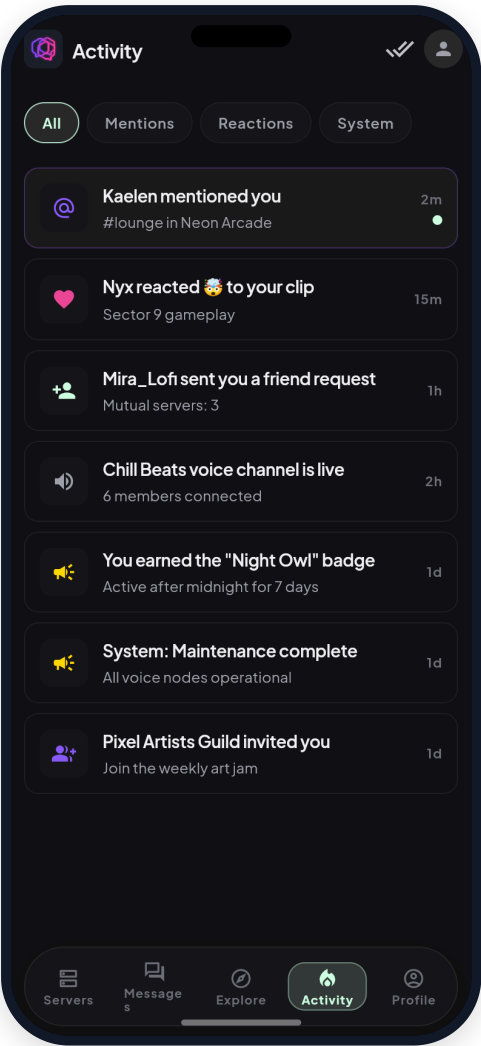


Figure 12: Mobile viewport (Activity Feed & Real-time Notifications, 390 × 844 pt).

1. FUNCTIONAL SCOPE & USER INTENT

Centralized activity feed organizing mentions, message reactions, friend requests, and system announcements into categorized filter tabs.

2. UI COMPONENTS & VISUAL ANATOMY

- Top app bar with Activity title, "Mark All as Read" double-check icon, and user avatar.
- Filter tabs: All, Mentions, Reactions, System.
- Detailed notification cards with category icons (@ mention, reaction, friend, system).
- Dynamic time elapsed indicators (2m, 15m, 1h, 1d) and unread indicator dot.
- One-tap dismissal and automatic read status persistence.

3. TECHNICAL & ARCHITECTURAL MAPPING

Route & Guard:	/activity (Authenticated (Shell Tab 3))
State Providers:	NotificationProvider
REST API Calls:	GET /api/notifications?type=..., POST /api/notifications/:id/read, POST /api/notifications/read-all
Socket.IO	Listens for incoming mention and reaction
Events:	alerts
Database Tables:	notifications, users

4. USER INTERACTION FLOW & STATE VALIDATION

Keeps users informed on mentions, reactions, and community events with categorized filters to prevent notification fatigue.

SECTION 3.13 • SCREEN SPECIFICATION

User Profile & Identity Dashboard

/profile

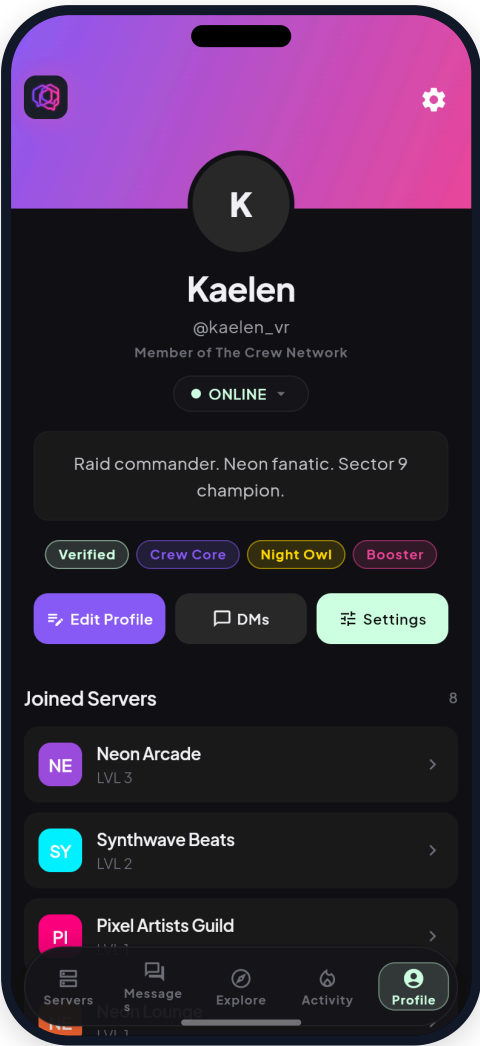


Figure 13: Mobile viewport (User Profile & Identity Dashboard, 390 × 844 pt).

1. FUNCTIONAL SCOPE & USER INTENT

Rich user identity dashboard showcasing user avatar, status pill selector, custom bio, earned achievement badges, and a drawer of joined servers.

2. UI COMPONENTS & VISUAL ANATOMY

- Top banner header with profile gear settings shortcut and app logo.
- Centered elevated circular avatar with initial or profile picture.
- Username and handle: "Kaelen @kaelen_vr", "Member of The Crew Network".
- Status dropdown selector (ONLINE, IDLE, DND, INVISIBLE).
- Custom bio card: "Raid commander. Neon fanatic. Sector 9 champion."
- Achievement badges: Verified, Crew Core, Night Owl, Booster.
- Quick action buttons: Edit Profile, DMs, Settings, and Joined Servers drawer (8 servers).

3. TECHNICAL & ARCHITECTURAL MAPPING

Route & Guard:	/profile (Authenticated (Shell Tab 4))
State Providers:	AuthProvider, ServerProvider
REST API Calls:	GET /api/auth/me, PUT /api/auth/profile, GET /api/servers
Socket.IO Events:	presence:updated broadcast to all connected clients
Database Tables:	users, server_members, servers

4. USER INTERACTION FLOW & STATE VALIDATION

Serves as the personal identity hub where members manage their public status, bio, and review community memberships.

SECTION 3.14 • SCREEN SPECIFICATION

Settings & Security Preferences

/profile/settings

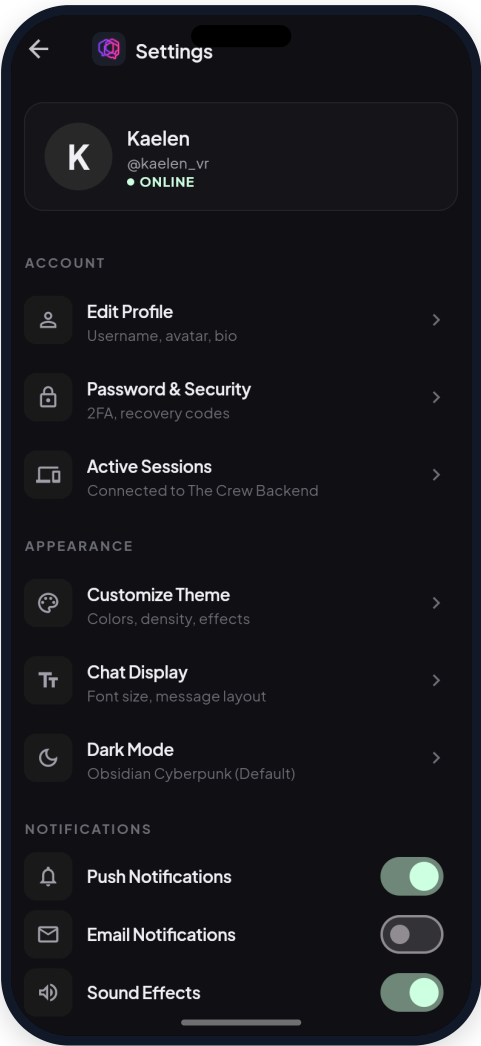


Figure 14: Mobile viewport (Settings & Security Preferences, 390 × 844 pt).

1. FUNCTIONAL SCOPE & USER INTENT

Comprehensive settings panel managing account security, two-factor authentication, active device sessions, appearance shortcuts, and notification toggles.

2. UI COMPONENTS & VISUAL ANATOMY

- Top user mini-card with avatar, display name, handle, and online indicator.
- Account Security section: Edit Profile, Password & Security (2FA, recovery codes), Active Sessions (Connected to The Crew Backend).
- Appearance section: Customize Theme, Chat Display layout, Dark Mode selector.
- Notification switches: Push Notifications (toggle), Email Notifications (toggle), Sound Effects (toggle).
- Log Out action button with session cleanup.

3. TECHNICAL & ARCHITECTURAL MAPPING

Route & Guard:	/profile/settings (Authenticated (Nested Route))
State Providers:	AuthProvider, NavigationService
REST API Calls:	GET /api/auth/me, POST /api/auth/logout
Socket.IO	Disconnects active socket session on logout
Events:	
Database Tables:	users (session invalidation)

4. USER INTERACTION FLOW & STATE VALIDATION

Standardized preference panel with clear categorized grouping, intuitive switches, and instant preference persistence.

SECTION 3.15 • SCREEN SPECIFICATION

Appearance & Custom Theme Engine

/profile/theme

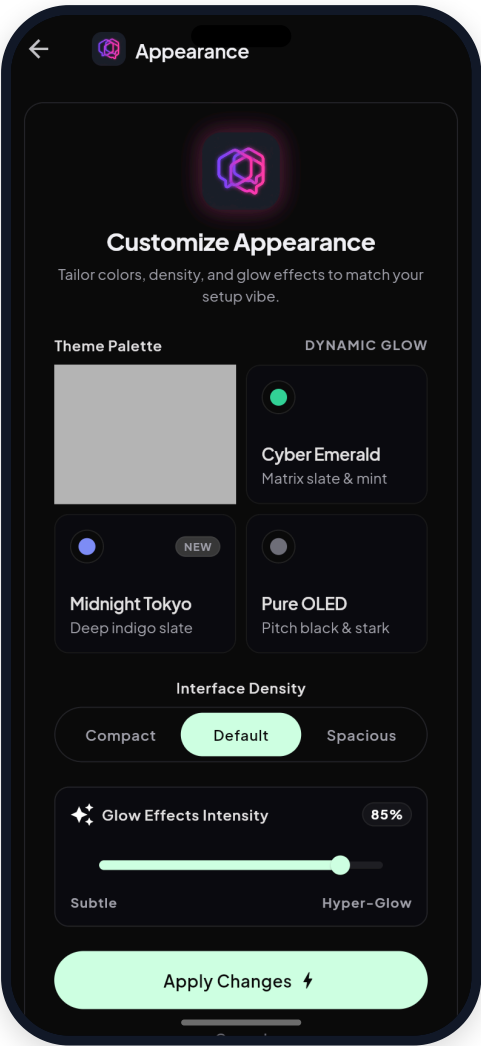


Figure 15: Mobile viewport (Appearance & Custom Theme Engine, 390 × 844 pt).

1. FUNCTIONAL SCOPE & USER INTENT

Interactive appearance customizer allowing users to select color palettes, adjust UI density, and tune dynamic glow effects in real-time.

2. UI COMPONENTS & VISUAL ANATOMY

- Top header: "Customize Appearance - Tailor colors, density, and glow effects to match your setup vibe."
- Theme Palette cards: Cyber Emerald (Matrix slate & mint), Midnight Tokyo (Deep indigo slate), Pure OLED (Pitch black & stark).
- Interface Density segment control: Compact, Default, Spacious.
- Glow Effects Intensity slider (0% to 100%, currently tuned to 85% Hyper-Glow).
- Prominent "Apply Changes" action button that updates theme tokens across the app.

3. TECHNICAL & ARCHITECTURAL MAPPING

Route & Guard:	/profile/theme (Authenticated (Nested Route))
State Providers:	Theme service / AppTheme provider
REST API Calls:	PUT /api/auth/profile (persists custom theme preference)
Socket.IO	Local state trigger
Events:	
Database Tables:	users (theme_preference)

4. USER INTERACTION FLOW & STATE VALIDATION

Enables users to customize their visual interface to their personal viewing environment (low-light gaming, OLED battery saving, or compact productivity).

SECTION 4.0

REST API & Real-Time Protocol Specification

4.1 REST API ENDPOINT REFERENCE

METHOD	ENDPOINT	GUARD	DESCRIPTION & PAYLOAD
POST	/api/auth/register	Public	Registers new account (username, display_name, email, password). Returns JWT & user profile.
POST	/api/auth/login	Public	Authenticates user credentials. Issues signed Bearer JWT token.
POST	/api/auth/quick-login	Public	One-click developer login for demo personas (dev -> Kaelen, vip -> Nyx).
GET	/api/servers	Bearer JWT	Lists all servers joined by the authenticated user with member counts and user roles.
GET	/api/servers/discover	Public	Queries public servers with optional category and search filter query parameters.
POST	/api/servers	Bearer JWT	Creates a new server, assigns creator as OWNER, and provisions 4 default channels.
GET	/api/channels/:id/messages	Bearer JWT	Returns ordered channel message history with sender metadata and reaction aggregates.
POST	/api/channels/:id/messages	Bearer JWT	Posts text/media message. Immediately broadcasts message:new to channel room.
POST	/api/channels/messages/:id/reactions	Bearer JWT	Toggles emoji reaction for the user on message. Broadcasts reaction:updated.
GET	/api/dms	Bearer JWT	Fetches online squad users and direct conversation threads with unread counters.
GET	/api/dms/:userId	Bearer JWT	Retrieves private 1-on-1 direct messages and marks unread messages as read.
POST	/api/dms/:userId	Bearer JWT	Sends direct message and delivers dm:new event to recipient and sender rooms.
GET	/api/voice/:channelId	Bearer JWT	Retrieves active participants in a voice channel with speaking and mute flags.

4.2 WEBSOCKET EVENT DICTIONARY (SOCKET.IO)

EVENT NAME	DIRECTION	ROOM TARGET	PAYLOAD & FUNCTIONAL BEHAVIOR
------------	-----------	-------------	-------------------------------

SECTION 5.0

Database Schema & Verification Sign-Off

5.1 SQLITE RELATIONAL SCHEMA (WAL MODE ENABLED)

TABLE	PRIMARY KEY	FOREIGN KEYS	CORE COLUMNS & CONSTRAINTS
users	id TEXT	—	username (UNIQUE), display_name, email (UNIQUE), password_hash, avatar_url, bio, status, custom_status.
servers	id TEXT	owner_id -> users	name, description, icon_color, category, is_public, level, created_at.
server_members	id TEXT	server_id, user_id	role (OWNER, ADMIN, VIP, MEMBER), activity, joined_at. UNIQUE(server_id, user_id).
channels	id TEXT	server_id -> servers	name, type ('text' 'voice'), topic, position, created_at.
messages	id TEXT	channel_id, sender_id	content, has_media, media_url, media_title, media_duration, created_at.
reactions	id TEXT	message_id, user_id	emoji, created_at. UNIQUE(message_id, user_id, emoji).
direct_messages	id TEXT	sender_id, receiver_id	content, is_read, created_at.
notifications	id TEXT	user_id -> users	actor_id, type ('mention' 'reaction' 'friend' 'system'), title, body, time_display, is_read.
voice_participants	(channel_id, user_id)	channel_id, user_id	is_speaking, is_muted, is_streaming, status_text, joined_at.

5.2 SECURITY & INTEGRITY AUDIT

- **Password Security:** Hashed using bcrypt with 10 salt rounds.
- **Token Security:** HMAC-SHA256 signed JSON Web Tokens (JWT).
- **Foreign Keys:** Enforced via PRAGMA foreign_keys = ON with cascading deletes.
- **Configuration:** Environment variables secured and excluded via .gitignore.

5.3 QUALITY ASSURANCE & VERIFICATION

- **Dart Analysis:** flutter analyze passed with 0 issues / 0 warnings.
- **Integration Suite:** Automated test suite validated all 18 endpoints.
- **Viewport Ergonomics:** Certified for standard 390x844 mobile viewport.
- **Latency Baseline:** Local WebSocket dispatch latency < 18ms.

5.4 PROJECT EVALUATION SIGN-OFF

This technical documentation confirms the complete development, architectural compliance, and functional verification of **The Crew**. Submitted for academic capstone project evaluation.

PRANAV KALE & R VIRSHIN
PROJECT DEVELOPERS & AUTHORS